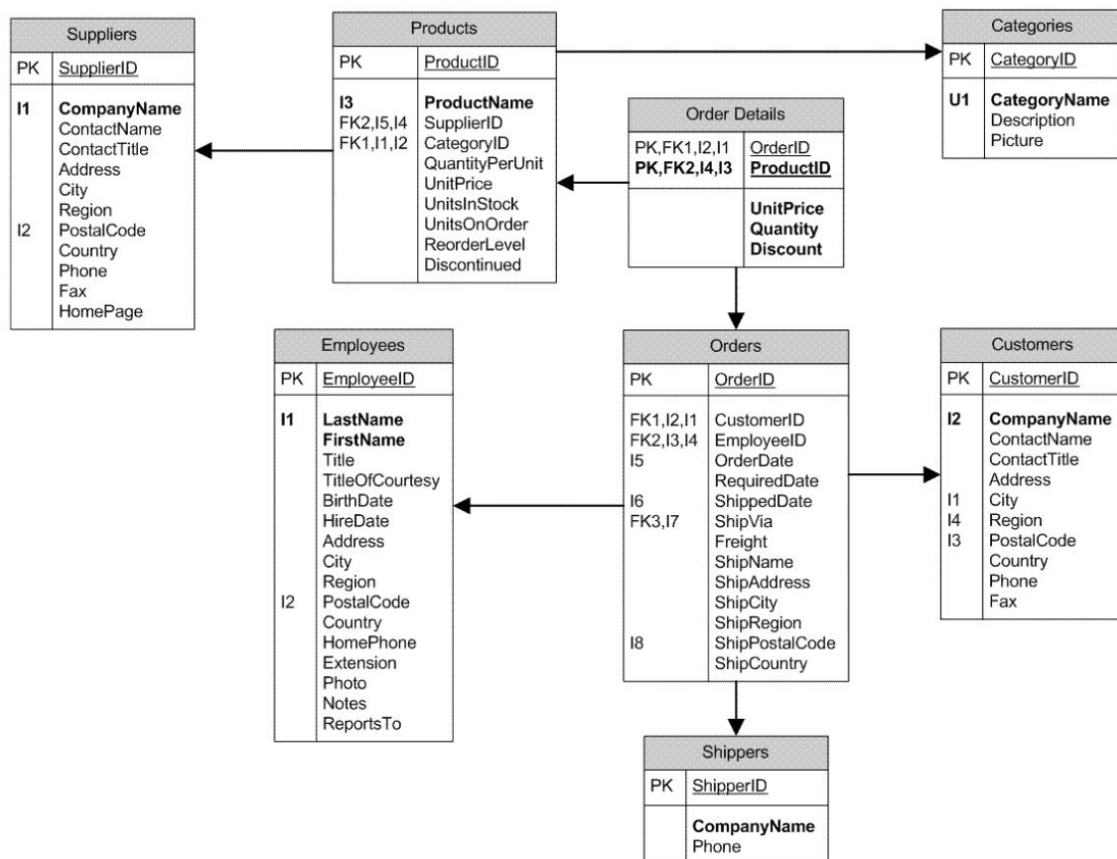


PRUEBA TECNICA ASISYA

DEV II

Se necesita exponer información de productos y categorías a clientes y procesos internos mediante una API robusta, escalable y segura. Se espera que la solución refleje no solo habilidades técnicas, sino también criterios arquitectónicos.



Objetivos

Desarrollar una solución API REST sobre el anterior modelo, implementando:

- API REST con operaciones CRUD básicas y consulta avanzada.
- Procesamiento masivo y eficiente de datos.
- Seguridad, escalabilidad y pruebas automatizadas.

Requerimientos Técnicos

1. Arquitectura

- El Back debe ser desarrollado en .net Core7 c# en adelante
- Diseñar bajo principios de arquitectura limpia (por ejemplo: Hexagonal, DDD o basada en capas).
- Usar DTOs y mapeo explícito (no exponer entidades directamente).
- Documentar las decisiones arquitectónicas (en el README).

2. Persistencia

- Crear e inicializar la base de datos relacional (MySQL o PostgreSQL recomendado).
- Cargar 100.000 productos de manera eficiente desde la API (`POST /Product`).
- Asociar productos a categorías "SERVIDORES" y "CLOUD", creadas vía `POST /Category`.

3. Endpoints requeridos

- POST /Category/ → crear categorías.
- POST /Product/ → generar y guardar productos aleatorios (debe soportar carga masiva).
- GET /Products/ → listar productos con paginación, filtros y búsqueda.
- GET /Products/{id} → detalle de producto con foto de la categoría.
- DELETE o PUT para actualizar/borrar productos.

4. Escalabilidad y Performance

- Soporte para cargas altas (estrategias como batch inserts, colas, procesamiento asíncrono, cache).
- Demostrar cómo escalarías horizontalmente tu solución en un entorno cloud (agregar en el readme)

5. Seguridad

- Implementar autenticación basada en JWT.
- Asegurar endpoints críticos.

6. Pruebas

- Incluir pruebas unitarias y al menos una prueba de integración.
- Mock de dependencias con herramientas como Mockito, Testcontainers, etc.

7. DevOps y despliegue

- la aplicación debe ir en Docker Proveer Dockerfile y docker-compose.yml.
- Integrar pipeline básico de CI (GitHub Actions, GitLab CI o similar).

8. Entrega

- Código en repositorio público (GitHub) .
- Instrucciones claras en README.md para:
- Clonar, construir y ejecutar localmente.

Notas adicionales

. CI/CD con GitHub Actions

Se debe incluir un archivo de pipeline mínimo en .github/workflows. Este debe permitir verificación automática del proyecto mediante tareas como: build, test, validación de estilo y (opcional) docker build. Esto podrá validarse desde la pestaña 'Actions' del repositorio en GitHub.

. Docker y ejecución local

Es obligatorio incluir un Dockerfile funcional.

Frontend – React JS Crear una SPA (Single Page Application)

Funcionalidades requeridas:

- Login con usuario y contraseña (obtener token JWT desde la API).
- Guardar token en `localStorage`.
- Implementar un interceptor para agregar el token a cada request.
- Crear un AuthGuard para proteger las rutas de productos.
- Página de listado de productos
- Formularios para crear y editar productos, con validaciones (Reactive Forms). - Implementar enrutamiento modular (AppRoutingModule)